研发过程记录

一、研发阶段总览

本项目采用迭代式开发方法，按照"核心流程先行、扩展功能跟进"的优先级组织研发工作。整体研发过程分为以下阶段：

```
阶段一：需求分析与方案设计          （2025年底）
阶段二：核心架构搭建与基础功能      （2026年初）
阶段三：分离引擎与指标体系建设      （2026年上半年）
阶段四：主观评价模块与统计分析      （2026年年中）
阶段五：系统完善与论文撰写          （2026年年中）
```

二、各阶段详细记录

阶段一：需求分析与方案设计

时间：2025年底

过程描述：

在自研音乐源分离算法的过程中，遇到了实验管理混乱的痛点。最初的想法来源于一个具体场景：当需要用多种算法对同一批音频进行分离测试时，需要手动管理不同算法的输入输出，用 Excel 手工记录指标数据，再用 Python 脚本画图对比——整个过程繁琐且容易出错。

于是产生了"做一个统一的实验管理平台"的想法。首先梳理了核心需求：

1. 需要管理什么数据？—— 混合音频、参考音轨、分离结果、客观指标、主观评分
2. 需要服务哪些用户？—— 训练模型的"训练者"和负责听感评估的"音频专家"
3. 核心流程是什么？—— 上传 → 分离 → 评测 → 分析 → 反馈

在这个阶段画出了第一版系统架构草图（见图1 问题解决逻辑图，见产品图纸材料），并确定了以下关键技术选型：

- 后端：Python FastAPI（异步能力、生态成熟）
- 前端：React（组件化、丰富的图表库）
- 数据库：MySQL（关系型数据需求、事务支持）
- 分离引擎：基于 ZFTurbo 的 Music-Source-Separation-Training 推理脚本

关键决策记录：

- 决策1：前后端分离架构 —— 原因：前端和后端可以独立演进，后续可扩展移动端或桌面端
- 决策2：分离引擎采用子进程调用而非进程内推理 —— 原因：避免 PyTorch 重依赖污染后端主进程，不同模型的依赖版本可能冲突
- 决策3：任务队列采用单任务串行处理 —— 原因：GPU 显存有限，同时跑多个分离任务容易 OOM，串行执行更稳定
- 决策4：使用雪花ID作为主键 —— 原因：分布式场景下的唯一ID生成，同时避免自增ID暴露数据规模

阶段二：核心架构搭建与基础功能

时间：2026年初

过程描述：

在方案确定后，开始搭建系统的基础架构。这个阶段的主要工作是"让系统跑起来"。

第一步：搭建后端项目骨架

```
AudioLab/backend/
├── app/
|   ├── main.py          # FastAPI 应用入口，CORS、路由注册
|   ├── database.py      # 异步 SQLAlchemy 引擎与会话工厂
|   ├── models.py        # ORM 数据模型定义
|   └── routers/         # API 路由模块
```

创建了初始的数据库模型设计（图3 总体架构图的数据层部分，见产品图纸材料），包括用户表、任务表、评分表、音频资源表等核心实体，以及它们之间的关联关系。采用了雪花ID生成器（Snowflake ID）作为全局唯一主键策略，以支持未来的分布式扩展需求。

遇到的技术问题：雪花ID在JavaScript端会因整数精度丢失导致ID解析错误。解决方案：API 响应中统一将 ID 字段序列化为字符串。

第二步：用户认证与权限体系

实现了 JWT 认证 + RBAC 权限控制。设计了三种角色：访客（可浏览）、专家（可评分）、管理员（可审批与管理）。注册流程包含邮箱验证码环节，专家注册需要管理员审批。

第三步：前端框架搭建

使用 Create React App 搭建前端项目，设计了主界面布局：左侧导航 + 右侧内容区。实现了基础的页面路由和登录注册页面。

阶段三：分离引擎与指标体系建设

时间：2026年上半年

过程描述：

这个阶段是项目的技术核心，主要解决"如何统一调用不同的分离算法"和"如何自动计算评估指标"两个关键问题。

第一步：分离引擎架构设计

经过多次讨论，确定了"策略模式"的设计方案：定义统一的 `SeparationEngine` 抽象基类，包含 `separate()` 方法，不同的引擎实现只需实现该方法即可。

最初只设计了一种引擎（MSSLocalEngine），直接调用 ZFTurbo 的 inference.py。后来为了支持前端联调测试（不需要真实 GPU 环境），增加了 MockSeparationEngine，用 ffmpeg 滤波器模拟分离效果。再后来为了支持远程部署场景，增加了 HttpSeparationEngine 和 LocalSeparationEngine。

引擎选择通过环境变量 `SEPARATION_ENGINE` 控制（mss/mock/http/local），在 `build_separation_engine_from_env()` 工厂函数中完成。

第二步：任务队列与调度

实现了单例模式的异步任务队列 `TaskQueueManager`，基于 asyncio 的 FIFO 队列。特性包括：单任务处理（同一时间只跑一个 GPU 任务）、队列位置查询、取消支持（从队列移除 + 进程终止）、CPU/GPU 切换。

关键技术细节：取消了是通过全局集合 `_CANCELLED_TASK_IDS` 实现软→硬中断。分离引擎在读取子进程输出时轮询该集合，如果发现已标记取消，则 terminate/kill 子进程以尽快释放 GPU。

第三步：客观指标计算

实现了两套指标计算方案：

- **双源方案**（`objective_metrics.py`）：假设 mixture = target + interference，通过最小二乘投影分解估计信号，计算 SDR/SIR/SAR/SI-SDR。适用于只有部分参考音轨的场景。
- **多源方案**（`objective_metrics_multisource.py`）：实现完整的 BSS Eval v3（Vincent et al. 2006），通过向各源子空间投影，分解估计信号为目标分量 + 空间失真 + 干扰误差 + 伪影误差。输出 SDR/SIR/SAR/ISR。

遇到的典型问题：Band Split RoFormer 的 checkpoint 与 config 维度不匹配（384 vs 192），导致 inference.py 报错 "size mismatch"。解决方案：在引擎的错误处理中增加了启发式检测逻辑，根据错误日志自动给出针对性的配置修改建议。

第四步：WOC/SOC 重叠一致性指标创新

在分析可视化结果时发现一个现象：有些分离结果的波形/频谱看起来和参考信号重叠得很好（肉眼判断），但 SDR 值并不高；另一些 SDR 值高的结果反而在某些频段有肉眼可见的泄漏。这说明客观指标和"看起来好不好"之间存在差异。

于是产生了设计一个"辅助解释指标"的想法：如果能将"重叠程度"数字化，就可以用它来桥接客观 SDR 和主观听感。经过推导，在 L1 归一化的幅度表示上定义了 IoU-like 的重叠一致性指标：

- **WOC（Waveform Overlap Consistency）**：基于时域波形幅度的重叠一致性
- **SOC（Spectrogram Overlap Consistency）**：基于 STFT 频谱幅度的重叠一致性

并进一步提出了三个扩展模块：相位一致性（PC）、时间对齐（Cross-Correlation）、感知加权（PSOC，基于 Mel/A-weighting）。

阶段四：主观评价模块与统计分析

时间：2026年年中

过程描述：

第一步：主观评分系统设计

参考 ITU-R BS.1534-1（MUSHRA）规范，设计了主观评价实验流程。评分采用双盲设计，评审员不知道所听音频对应哪种算法。评分维度有三个：音质（Quality）、干扰抑制（Interference）、伪影控制（Artifacts），采用 1-5 分李克特量表。

在技术实现上，设计了批量提交机制：一个任务的所有 4 个 stem（Vocals/Drums/Bass/Other）的评分必须同时提交，确保评分数据的完整性。

第二步：统计分析方法实现

为了不引入 SciPy 重依赖，从零实现了以下统计方法：

- **Spearman 等级相关系数**（含平均秩处理打结数据）
- **排列检验（Permutation Test）**：用于计算 SOC 与主观音质评分之间相关性的显著性

- **Wilcoxon 符号秩检验**（含正态近似、打结修正、秩双列效应量）：用于两两算法差异的显著性检验

这些方法被集成到 API 端点中，用户（评审员/研究者）可以在 Web 界面上直接触发统计分析并查看结果。

第三步：主客观联合分析

实现了主客观数据的联合查询（`subjective-objective-points` 端点），将同一分离任务的客观指标与主观评分关联起来，支持按 stem、按算法、按维度进行分组分析。实现了 SOC 与音质评分的 Spearman 相关 + 排列检验的完整分析链路（`soc-quality-correlation` 端点）。

阶段五：系统完善与论文撰写

时间：2026年年中

过程描述：

在核心功能稳定后，进入完善和文档化阶段：

1. **系统管理功能完善**：添加了系统日志审计、系统配置管理（键值对配置项，支持分类）、通知中心（支持定向推送和过期管理）、数据统计仪表盘（用户增长、任务统计、资源统计等）。
2. **音频资源库完善**：实现了公共音频资源的上传、浏览、下载、分类过滤功能，以及捆绑上传（Mixture + 4 个 Gold Stems 打包上传）。
3. **专家反馈渠道**：实现了专家与管理员之间的消息系统，专家可以提交改进意见，管理员可以回复。
4. **实验设计与数据收集**：基于 MUSDB18 数据集（150 首歌曲，44.1kHz 立体声），设计了完整的对比实验方案。选取 MDX23C、Demucs4HT、Band Split RoFormer、SCNet 四种代表性算法，在 50 首测试集歌曲上进行了系统性的客观指标对比。同时邀请 5 名音频专家对 10 首歌曲的分离结果进行了主观评分。
5. **论文撰写**：基于实验结果和系统设计，撰写了《AudioLab——基于主客观评估的音乐源分离系统》学术论文，系统性地阐述了主客观协同评估方法、系统架构设计、WOC/SOC 辅助指标及其统计验证方案。

三、研发过程中的关键草图与设计笔记

3.1 初期需求梳理（草图阶段）

在项目初期，为了理清"谁做什么、什么顺序做"，在白板上画出了用户故事地图：

- 横轴：时间顺序（注册 → 上传 → 分离 → 评测 → 分析 → 反馈）
- 纵轴：角色分层（访客层 / 专家层 / 管理员层 / 算法工程师层）
- 每个格子：该角色在该阶段需要完成的具体操作

这个用户故事地图后来被正式绘制为系统的用户故事地图（见图5，见产品图纸材料），成为后续功能开发优先级排序的依据。

3.2 核心算法流程设计草图

为了统一不同分离算法的调用流程，在纸上画出了标准化的算法流水线：

```
输入音频 → 预处理(重采样/声道规整) → STFT → 模型推理 → 逆STFT → 后处理(归一化/限幅) → 输出音频
```

这个流水线设计后来被正式绘制为音乐源分离算法流程图（见图6，见产品图纸材料）。

3.3 WOC/SOC 指标的初步推导笔记

在分析可视化结果时注意到"重叠看起来好但 SDR 不高"的现象后，在笔记本上做了以下推导（原始笔记转录）：

1. 如果把参考信号和估计信号的幅度都归一化到 [0, 1] 区间
2. 那么它们的"交集"（min 操作）反映了共同成分
3. "并集"（max 操作）反映了总成分
4. 交集/并集的比例就衡量了"重叠程度"
5. 这个想法在时域（WOC）和时频域（SOC）都适用

随后在 Python 中实现了初版算法并验证了可行性，最终形式化为论文中的数学定义。

四、研发资料索引

材料类型	文件名	说明
架构图	图1 问题解决逻辑图	系统要解决的三个问题及其对应方案
架构图	图2 训练者与专家协作图	两类核心角色的协作关系
架构图	图3 总体架构图	四层解耦架构（服务/业务/技术/数据）
流程图	图4 业务流程图	从训练到反馈的完整闭环
规划图	图5 用户故事地图	按角色和时间的功能规划
算法图	图6 音乐源分离算法流程图	标准化分离流水线
代码库	backend/	后端完整源码
代码库	frontend/	前端完整源码
论文	AudioLab--基于主客观评估的音乐源分离系统.md	系统设计与实验论文